



university of
 groningen

Models and Semantics of Computation (BaSC)

Lecture 7: Denotational semantics of IMP

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

Lambda Notation: Key Ingredients



Anonymous functions

$$\lambda x. e$$

- ▶ a function that waits for a value to be substituted for x and then evaluates e

Applications

$$e_1 e_2$$

- ▶ denotes the application of the function e_1 to argument e_2

Lambda Notation: Key Ingredients



Anonymous functions

$$\lambda x. e$$

- a function that waits for a value to be substituted for x and then evaluates e

Applications

$$e_1 e_2$$

- denotes the application of the function e_1 to argument e_2

Conditional

$$e \blacktriangleright (e_1 ; e_2)$$

- (In the book: $e \rightarrow e_1, e_2.$) The meaning is as usual: if e then e_1 else e_2 .
- Example: $\min \triangleq \lambda x. \lambda y. x < y \blacktriangleright (x ; y)$

Denotational Semantics of IMP



An abstract, purely mathematical semantics to IMP.

- ▶ The **operational semantics** gives an executable perspective: given a program and a state, we derive the state obtained after the program's execution.
- ▶ The **denotational semantics** takes a program and returns its associated transformation function over memories.

Denotational Semantics of IMP



An abstract, purely mathematical semantics to IMP.

- ▶ The **operational semantics** gives an executable perspective: given a program and a state, we derive the state obtained after the program's execution.
- ▶ The **denotational semantics** takes a program and returns its associated transformation function over memories.

Alternatively:

- ▶ Operational semantics: An “**interpreter**” of the language (input: program and initial state; output: final state after execution)
- ▶ Denotational semantics: A “**compiler**” for the language, from commands to functional programs (in the more abstract lambda notation).

Denotational Semantics of IMP



We use lambda notation as a meta-language to express the semantics of IMP.
We define three **interpretation functions** that map IMP syntax into lambda terms.

$\mathcal{A}[\cdot] : \text{AExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$ (functions from memories to integers)

$\mathcal{B}[\cdot] : \text{BExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$ (functions from memories to booleans)

Denotational Semantics of IMP



We use lambda notation as a meta-language to express the semantics of IMP.
We define three **interpretation functions** that map IMP syntax into lambda terms.

$\mathcal{A}[\cdot] : \text{AExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$ (functions from memories to integers)

$\mathcal{B}[\cdot] : \text{BExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$ (functions from memories to booleans)

$\mathcal{C}[\cdot] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M})$ (partial functions from memories to memories)

Denotational Semantics: Arith Expressions



$$\mathcal{A}[\![\cdot]\!] : \text{AExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$$

Notation Given an arithmetic expression a , we write

$$\mathcal{A}[\![n]\!]\sigma \triangleq a$$

rather than

$$\mathcal{A}[\![a]\!] \triangleq \lambda\sigma. a$$

Denotational Semantics: Arith Expressions



$$\mathcal{A}[\![\cdot]\!] : \text{AExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{Z})$$

Notation Given an arithmetic expression a , we write

$$\mathcal{A}[\![n]\!]\sigma \triangleq a$$

rather than

$$\mathcal{A}[\![a]\!] \triangleq \lambda\sigma. a$$

Definition by structural induction on the syntax of arithmetic expressions:

$$\mathcal{A}[\![n]\!]\sigma \triangleq n$$

[the constant function $\lambda\sigma. n$]

$$\mathcal{A}[\![x]\!]\sigma \triangleq \sigma(x)$$

[the function $\lambda\sigma. \sigma(x)$]

$$\mathcal{A}[\![a_0 \text{ op } a_1]\!]\sigma \triangleq \mathcal{A}[\![a_0]\!]\sigma \text{ op } \mathcal{A}[\![a_1]\!]\sigma$$



$$\mathcal{B}[\![\cdot]\!] : \text{BExp} \rightarrow (\mathbb{M} \rightarrow \mathbb{B})$$

Similarly as before, we define:

$$\mathcal{B}[\![v]\!]\sigma \triangleq v$$

$$\mathcal{B}[\![\neg b]\!]\sigma \triangleq \neg \mathcal{B}[\![b]\!]\sigma$$

$$\mathcal{B}[\![a_0 \text{ cmp } a_1]\!]\sigma \triangleq \mathcal{A}[\![a_0]\!]\sigma \text{ cmp } \mathcal{A}[\![a_1]\!]\sigma$$

$$\mathcal{B}[\![b_0 \text{ bop } b_1]\!]\sigma \triangleq \mathcal{B}[\![b_0]\!]\sigma \text{ bop } \mathcal{B}[\![b_1]\!]\sigma$$

Equivalence for Arithmetic Expressions



A natural question:

$$\forall a, \sigma, n \quad \langle a, \sigma \rangle \longrightarrow n \quad \stackrel{?}{\iff} \quad \mathcal{A}[[a]]\sigma = n$$

Equivalence for Arithmetic Expressions



A natural question:

$$\forall a, \sigma, n \quad \langle a, \sigma \rangle \longrightarrow n \quad \stackrel{?}{\iff} \quad \mathcal{A}[[a]]\sigma = n$$

Formally:

For all arithmetic expressions $a \in \text{AExp}$, the following predicate $P(a)$ holds

$$P(a) \triangleq \forall \sigma \in \mathbb{M}. \langle a, \sigma \rangle \longrightarrow \mathcal{A}[[a]]\sigma$$

How to proceed?

Equivalence for Arithmetic Expressions



A natural question:

$$\forall a, \sigma, n \quad \langle a, \sigma \rangle \longrightarrow n \quad \stackrel{?}{\iff} \quad \mathcal{A}[[a]]\sigma = n$$

Formally:

For all arithmetic expressions $a \in \text{AExp}$, the following predicate $P(a)$ holds

$$P(a) \triangleq \forall \sigma \in \mathbb{M}. \langle a, \sigma \rangle \longrightarrow \mathcal{A}[[a]]\sigma$$

How to proceed? By **structural induction**!

Base Cases



$$\forall x \in \text{Ide. } P(x)$$

- ▶ Take any $x \in \text{Ide.}$ We need to prove $P(x) \triangleq \forall \sigma. \langle x, \sigma \rangle \longrightarrow \mathcal{A}[[x]]\sigma = \sigma(x)$.
- ▶ **Proof:** Take an arbitrary σ and conclude by rule $\frac{}{\langle x, \sigma \rangle \longrightarrow \sigma(x)}$.

Base Cases



$$\forall x \in \text{Ide. } P(x)$$

- ▶ Take any $x \in \text{Ide.}$ We need to prove $P(x) \triangleq \forall \sigma. \langle x, \sigma \rangle \longrightarrow \mathcal{A}[[x]]\sigma = \sigma(x)$.
- ▶ **Proof:** Take an arbitrary σ and conclude by rule $\frac{}{\langle x, \sigma \rangle \longrightarrow \sigma(x)}$.

$$\forall n \in \mathbb{Z}. P(n)$$

- ▶ Take any $n \in \mathbb{Z}.$ We need to prove $P(n) \triangleq \forall \sigma. \langle n, \sigma \rangle \longrightarrow \mathcal{A}[[n]]\sigma = n$.
- ▶ **Proof:** Take an arbitrary σ and conclude by rule $\frac{}{\langle n, \sigma \rangle \longrightarrow n}$.

Inductive Case



$$\forall a_0, a_1. P(a_0) \wedge P(a_1) \Rightarrow P(a_0 \text{ op } a_1)$$

Take arbitrary a_0, a_1 . We assume $P(a_i) \triangleq \forall \sigma. \langle a_i, \sigma \rangle \longrightarrow \mathcal{A}[[a_i]]\sigma$ ($i \in \{0, 1\}$).
To prove:

$$\begin{aligned} P(a_0 \text{ op } a_1) &\triangleq \forall \sigma. \langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow \mathcal{A}[[a_0 \text{ op } a_1]]\sigma \\ &= \mathcal{A}[[a_0]]\sigma \text{ op } \mathcal{A}[[a_1]]\sigma \end{aligned}$$

Inductive Case



$$\forall a_0, a_1. P(a_0) \wedge P(a_1) \Rightarrow P(a_0 \text{ op } a_1)$$

Take arbitrary a_0, a_1 . We assume $P(a_i) \triangleq \forall \sigma. \langle a_i, \sigma \rangle \longrightarrow \mathcal{A}[[a_i]]\sigma$ ($i \in \{0, 1\}$).
To prove:

$$\begin{aligned} P(a_0 \text{ op } a_1) &\triangleq \forall \sigma. \langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow \mathcal{A}[[a_0 \text{ op } a_1]]\sigma \\ &= \mathcal{A}[[a_0]]\sigma \text{ op } \mathcal{A}[[a_1]]\sigma \end{aligned}$$

► Take an arbitrary σ . We have:

$$\langle a_0 \text{ op } a_1, \sigma \rangle \longrightarrow n \nwarrow_{n=n_0 \text{ op } n_1} \langle a_0, \sigma \rangle \longrightarrow n_0, \langle a_1, \sigma \rangle \longrightarrow n_1$$

► By IH, $n_i = \mathcal{A}[[a_i]]\sigma$. Hence, $n = n_0 \text{ op } n_1 = \mathcal{A}[[a_0]]\sigma \text{ op } \mathcal{A}[[a_1]]\sigma$.

Equivalence for Boolean Expressions



For all boolean expressions $b \in \text{BExp}$, the following predicate $P(b)$ holds

$$P(b) \triangleq \forall \sigma \in \mathbb{M}. \langle b, \sigma \rangle \longrightarrow \mathcal{B} \llbracket b \rrbracket \sigma$$

How to proceed? Another **structural induction** (left as an exercise).

The Case of Commands



Defining the denotational semantics for commands needs additional machinery.
The key issue is **divergence**.

The Case of Commands



Defining the denotational semantics for commands needs additional machinery.
The key issue is **divergence**.

- In the interpretation for commands, we must consider **partial functions**:

$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M})$$

This is because executing a command may not lead to a memory.

The Case of Commands



Defining the denotational semantics for commands needs additional machinery.
The key issue is **divergence**.

- In the interpretation for commands, we must consider **partial functions**:

$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M})$$

This is because executing a command may not lead to a memory.

- Let $\perp$ ('bottom') denote divergence. Define $\mathbb{M}_\perp \triangleq \mathbb{M} \cup \{\perp\}$.

We can then proceed with a total function instead:

$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M}_\perp)$$

The Case of Commands



- Because of divergence, handling sequential composition and while is delicate.

The Case of Commands



- ▶ Because of divergence, handling sequential composition and while is delicate.
- ▶ Consider composition: to define $\mathcal{C}[[c_0; c_1]]\sigma$, we would like to
 - (i) obtain a denotation f for c_0 , and
 - (ii) use f to obtain a denotation for c_1 .

The Case of Commands



- ▶ Because of divergence, handling sequential composition and while is delicate.
- ▶ Consider composition: to define $\mathcal{C}[[c_0; c_1]]\sigma$, we would like to
 - (i) obtain a denotation f for c_0 , and (ii) use f to obtain a denotation for c_1 .
- ▶ **Problem:** what to do if c_0 diverges, and never produces a memory σ ?

The Case of Commands



- ▶ Because of divergence, handling sequential composition and while is delicate.
- ▶ Consider composition: to define $\mathcal{C}[[c_0; c_1]]\sigma$, we would like to
 - (i) obtain a denotation f for c_0 , and (ii) use f to obtain a denotation for c_1 .
- ▶ **Problem:** what to do if c_0 diverges, and never produces a memory σ ?
- ▶ To account for the possibility of $\perp$, we use the **lifting** of a function.

The Case of Commands



- ▶ Because of divergence, handling sequential composition and while is delicate.
- ▶ Consider composition: to define $\mathcal{C}[[c_0; c_1]]\sigma$, we would like to
 - (i) obtain a denotation f for c_0 , and (ii) use f to obtain a denotation for c_1 .
- ▶ **Problem:** what to do if c_0 diverges, and never produces a memory σ ?
- ▶ To account for the possibility of $\perp$, we use the **lifting** of a function.
- ▶ Given a function $f : \mathbb{M} \rightarrow \mathbb{M}_\perp$, we write f^* to denote the function:

$$f^*(x) = \begin{cases} \perp & \text{if } x = \perp \\ f(x) & \text{otherwise} \end{cases}$$

This way, $f^* : \mathbb{M}_\perp \rightarrow \mathbb{M}_\perp$: it is just like f but can take $\perp$ as argument.

The Case of Commands



$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M}_\perp)$$

$$\mathcal{C}[\![\text{skip}]\!]\sigma \triangleq \sigma$$

$[\lambda\sigma. \sigma : \text{identity function}]$

The Case of Commands



$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M}_\perp)$$

$$\mathcal{C}[\![\text{skip}]\!]\sigma \triangleq \sigma \quad [\lambda\sigma. \sigma : \text{identity function}]$$

$$\mathcal{C}[\![x := a]\!]\sigma \triangleq \sigma[\mathcal{A}[a]\sigma/x]$$

$$\mathcal{C}[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!]\sigma \triangleq \begin{cases} \mathcal{C}[\![c_0]\!]\sigma & \text{if } \mathcal{B}[b]\sigma \\ \mathcal{C}[\![c_1]\!]\sigma & \text{otherwise} \end{cases}$$

The Case of Commands



$$\mathcal{C}[\![\cdot]\!] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M}_\perp)$$

$$\mathcal{C}[\![\text{skip}]\!]\sigma \triangleq \sigma \quad [\lambda\sigma. \sigma : \text{identity function}]$$

$$\mathcal{C}[\![x := a]\!]\sigma \triangleq \sigma[\mathcal{A}[a]\sigma/x]$$

$$\mathcal{C}[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!]\sigma \triangleq \begin{cases} \mathcal{C}[\![c_0]\!]\sigma & \text{if } \mathcal{B}[b]\sigma \\ \mathcal{C}[\![c_1]\!]\sigma & \text{otherwise} \end{cases}$$

$$\mathcal{C}[\![c_0; c_1]\!]\sigma \triangleq \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma)$$

$\uparrow$ c_0 may diverge, we need the lifted $\mathcal{C}[\![c_1]\!]$

The Case of Commands



$$\mathcal{C}[\cdot] : \text{Com} \rightarrow (\mathbb{M} \rightarrow \mathbb{M}_\perp)$$

$$\mathcal{C}[\text{skip}]\sigma \triangleq \sigma \quad [\lambda\sigma. \sigma : \text{identity function}]$$

$$\mathcal{C}[x := a]\sigma \triangleq \sigma[\mathcal{A}[a]\sigma/x]$$

$$\mathcal{C}[\text{if } b \text{ then } c_0 \text{ else } c_1]\sigma \triangleq \begin{cases} \mathcal{C}[c_0]\sigma & \text{if } \mathcal{B}[b]\sigma \\ \mathcal{C}[c_1]\sigma & \text{otherwise} \end{cases}$$

$$\mathcal{C}[c_0; c_1]\sigma \triangleq \mathcal{C}[c_1]^*(\mathcal{C}[c_0]\sigma)$$

$\uparrow$ c_0 may diverge, we need the lifted $\mathcal{C}[c_1]$

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \begin{cases} \sigma & \text{if } \neg \mathcal{B}[b]\sigma \\ \mathcal{C}[\text{while } b \text{ do } c]^*(\mathcal{C}[c]\sigma) & \text{otherwise} \end{cases}$$

Reasonable, but not a well-founded definition!

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq$$

$\Gamma_{b,c}$

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq$$

$$\Gamma_{b,c}$$

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**.

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

- Complete partial orders (CPOs), denoted $(D, \sqsubseteq)$.
(A PO is complete when each of its *chains* has a *limit*.)

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

- Complete partial orders (CPOs), denoted $(D, \sqsubseteq)$.
(A PO is complete when each of its *chains* has a *limit*.)
- Given $f : D \rightarrow D$, we say $d \in D$ is a **fixpoint** of f if $f(d) = d$.

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

- ▶ Complete partial orders (CPOs), denoted $(D, \sqsubseteq)$.
(A PO is complete when each of its *chains* has a *limit*.)
- ▶ Given $f : D \rightarrow D$, we say $d \in D$ is a **fixpoint** of f if $f(d) = d$.
- ▶ Given a CPO on D , we want **monotone**, **continuous** functions $f : D \rightarrow D$.

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

- Complete partial orders (CPOs), denoted $(D, \sqsubseteq)$.
(A PO is complete when each of its *chains* has a *limit*.)
- Given $f : D \rightarrow D$, we say $d \in D$ is a **fixpoint** of f if $f(d) = d$.
- Given a CPO on D , we want **monotone**, **continuous** functions $f : D \rightarrow D$.
- **Kleene**: The least fixpoint (wrt $\sqsubseteq$) of a continuous function exists (and is computable).

Denotational Semantics for While



A function defined inductively on b, c (see next)

$$\mathcal{C}[\text{while } b \text{ do } c]\sigma \triangleq \text{fix } \Gamma_{b,c}$$

Least fixpoint

$$\Gamma_{b,c} : (\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp$$

A **recursive equation**! We define a **fixpoint calculation**. Ingredients:

- ▶ Complete partial orders (CPOs), denoted $(D, \sqsubseteq)$.
(A PO is complete when each of its *chains* has a *limit*.)
- ▶ Given $f : D \rightarrow D$, we say $d \in D$ is a **fixpoint** of f if $f(d) = d$.
- ▶ Given a CPO on D , we want **monotone**, **continuous** functions $f : D \rightarrow D$.
- ▶ **Kleene**: The least fixpoint (wrt $\sqsubseteq$) of a continuous function exists (and is computable).
- ▶ The **immediate consequence operator** of a logical system R (and its fixpoint).

Denotational Semantics for While



- What is $\Gamma_{b,c}$? It takes a function $\varphi : \mathbb{M} \rightarrow \mathbb{M}_\perp$ and returns the function

$$\lambda\sigma. \mathcal{B}[[b]]\sigma \triangleright (\varphi^*(\mathcal{C}[[c]]\sigma); \sigma)$$

which acts depending on $\mathcal{B}[[b]]\sigma$.

Denotational Semantics for While



- What is $\Gamma_{b,c}$? It takes a function $\varphi : \mathbb{M} \rightarrow \mathbb{M}_\perp$ and returns the function

$$\lambda\sigma. \mathcal{B}[[b]]\sigma \triangleright (\varphi^*(\mathcal{C}[[c]]\sigma); \sigma)$$

which acts depending on $\mathcal{B}[[b]]\sigma$.

- It is typed as follows:

$$\Gamma_{b,c} \triangleq \lambda\varphi. \lambda\sigma. \underbrace{\underbrace{\underbrace{\mathcal{B}[[b]]\sigma \triangleright (\varphi^*(\mathcal{C}[[c]]\sigma); \sigma)}_{\mathbb{M}_\perp}}_{\mathbb{M} \rightarrow \mathbb{M}_\perp}}_{(\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp}$$

Denotational Semantics for While



- What is $\Gamma_{b,c}$? It takes a function $\varphi : \mathbb{M} \rightarrow \mathbb{M}_\perp$ and returns the function

$$\lambda\sigma. \mathcal{B}[[b]]\sigma \triangleright (\varphi^*(\mathcal{C}[[c]]\sigma); \sigma)$$

which acts depending on $\mathcal{B}[[b]]\sigma$.

- It is typed as follows:

$$\begin{array}{c} \Gamma_{b,c} \triangleq \lambda\varphi. \lambda\sigma. \underbrace{\mathcal{B}[[b]]\sigma \triangleright (\varphi^*(\mathcal{C}[[c]]\sigma); \sigma)}_{\mathbb{M}_\perp} \\ \underbrace{\hspace{10em}}_{\mathbb{M} \rightarrow \mathbb{M}_\perp} \\ \underbrace{\hspace{10em}}_{(\mathbb{M} \rightarrow \mathbb{M}_\perp) \rightarrow \mathbb{M} \rightarrow \mathbb{M}_\perp} \end{array}$$

- We require that $\mathcal{C}[[\text{while } b \text{ do } c]]\sigma$ is a **least fixpoint** of $\Gamma_{b,c}$:

$$\mathcal{C}[[\text{while } b \text{ do } c]]\sigma = \text{fix } \Gamma_{b,c}$$

Example 1



Given **while** tt **do skip**, we have

$$\mathcal{C}[\text{while } \text{tt} \text{ do skip}] \sigma = \text{fix } \Gamma_{\text{tt}, \text{skip}}$$

where

$$\begin{aligned} \Gamma_{\text{tt}, \text{skip}} \varphi \sigma &= \mathcal{B}[\text{tt}] \sigma \blacktriangleright (\varphi^*(\mathcal{C}[\text{skip}] \sigma); \sigma) \\ &= \text{tt} \blacktriangleright (\varphi^*(\mathcal{C}[\text{skip}] \sigma); \sigma) \\ &= \varphi^*(\mathcal{C}[\text{skip}] \sigma) \\ &= \varphi^* \sigma \\ &= \varphi \sigma \quad [\text{We know that } \sigma \neq \perp] \end{aligned}$$

Hence, $\Gamma_{\text{tt}, \text{skip}} \varphi = \varphi$, and so $\Gamma_{\text{tt}, \text{skip}}$ is the identity function.

In this case, every function is a fixpoint (although we want the least one).

Example 2



Consider $c \triangleq \text{while } \underbrace{x \neq 0}_b \text{ do } \underbrace{x := x - 1}_{c'}$. Then $\mathcal{C}[[c]]\sigma = \text{fix } \Gamma_{x \neq 0, c'}$, where

$$\begin{aligned}\Gamma_{b, c'} &= \lambda\sigma. \mathcal{B}[[b]]\sigma \blacktriangleright (\varphi^*(\mathcal{C}[[c']]\sigma); \sigma) \\ &= \lambda\sigma. \sigma x \neq 0 \blacktriangleright (\varphi^*(\sigma[\sigma x - 1/x]); \sigma) \\ &= \lambda\sigma. \sigma x \neq 0 \blacktriangleright (\varphi(\sigma[\sigma x - 1/x]); \sigma)\end{aligned}$$

Example 2



Consider $c \triangleq \text{while } \underbrace{x \neq 0}_b \text{ do } \underbrace{x := x - 1}_{c'}$. Then $\mathcal{C}[[c]]\sigma = \text{fix } \Gamma_{x \neq 0, c'}$, where

$$\begin{aligned}\Gamma_{b, c'} &= \lambda\sigma. \mathcal{B}[[b]]\sigma \blacktriangleright (\varphi^*(\mathcal{C}[[c']]\sigma); \sigma) \\ &= \lambda\sigma. \sigma x \neq 0 \blacktriangleright (\varphi^*(\sigma[\sigma x - 1/x]); \sigma) \\ &= \lambda\sigma. \sigma x \neq 0 \blacktriangleright (\varphi(\sigma[\sigma x - 1/x]); \sigma)\end{aligned}$$

By Kleene's theorem, the fixpoint of $\Gamma_{b, c'}$ can be computed. We have:

$$\mathcal{C}[[c]]\sigma = \lambda\sigma. 0 \leq \sigma x \blacktriangleright (\sigma[0/x]; \perp)$$

Equivalence for Commands / Consistency



For all commands $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, the following holds

$$\langle c, \sigma \rangle \longrightarrow \sigma' \iff \mathcal{C}[[c]]\sigma = \sigma'$$

How to proceed? We divide the property as follows:

Correctness

For all $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, we use ?? to prove

$$P(\langle c, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[[c]]\sigma = \sigma'$$

Completeness

For all $c \in \text{Com}$, we use ?? to prove

$$P(c) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[[c]]\sigma = \sigma' \Rightarrow \langle c, \sigma \rangle \longrightarrow \sigma')$$

Equivalence for Commands / Consistency



For all commands $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, the following holds

$$\langle c, \sigma \rangle \longrightarrow \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$$

How to proceed? We divide the property as follows:

Correctness

For all $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, we use **rule induction** to prove

$$P(\langle c, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c]\!]\sigma = \sigma'$$

Completeness

For all $c \in \text{Com}$, we use **??** to prove

$$P(c) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c]\!]\sigma = \sigma' \Rightarrow \langle c, \sigma \rangle \longrightarrow \sigma')$$

Equivalence for Commands / Consistency



For all commands $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, the following holds

$$\langle c, \sigma \rangle \longrightarrow \sigma' \iff \mathcal{C}[\![c]\!]\sigma = \sigma'$$

How to proceed? We divide the property as follows:

Correctness

For all $c \in \text{Com}$ and $\forall \sigma, \sigma' \in \mathbb{M}$, we use **rule induction** to prove

$$P(\langle c, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c]\!]\sigma = \sigma'$$

Completeness

For all $c \in \text{Com}$, we use **structural induction** to prove

$$P(c) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c]\!]\sigma = \sigma' \Rightarrow \langle c, \sigma \rangle \longrightarrow \sigma')$$

Correctness: Skip



$$\overline{\langle \text{skip}, \sigma \rangle \longrightarrow \sigma}$$

We want to prove

$$P(\langle \text{skip}, \sigma \rangle \longrightarrow \sigma) \triangleq \mathcal{C}[\![\text{skip}]\!]\sigma = \sigma$$

which is immediate by definition of the denotational semantics.

Correctness: Assignment



$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]}$$

- We assume $\langle a, \sigma \rangle \longrightarrow n$ and hence $\mathcal{A}[[a]]\sigma = n$ by the equivalence of the operational and denotational semantics for arithmetic expressions.

Correctness: Assignment



$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]}$$

- We assume $\langle a, \sigma \rangle \longrightarrow n$ and hence $\mathcal{A}[[a]]\sigma = n$ by the equivalence of the operational and denotational semantics for arithmetic expressions.
- We want to prove

$$P(\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]) \triangleq \mathcal{C}[[x := a]]\sigma = \sigma[n/x]$$



$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]}$$

- ▶ We assume $\langle a, \sigma \rangle \longrightarrow n$ and hence $\mathcal{A}[a]\sigma = n$ by the equivalence of the operational and denotational semantics for arithmetic expressions.
- ▶ We want to prove

$$P(\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]) \triangleq \mathcal{C}[x := a]\sigma = \sigma[n/x]$$

- ▶ By definition of the denotational semantics $\mathcal{C}[x := a]\sigma = \sigma[\mathcal{A}[a]\sigma/x]$.
Therefore,

$$\mathcal{C}[x := a]\sigma = \sigma[n/x]$$

Correctness: Composition



$$\frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'}$$

- We assume

$$P(\langle c_0, \sigma \rangle \longrightarrow \sigma'') \triangleq \mathcal{C}[\![c_0]\!]\sigma = \sigma''$$

$$P(\langle c_1, \sigma'' \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_1]\!]\sigma'' = \sigma'$$

- We want to prove $P(\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_0; c_1]\!]\sigma = \sigma' .$

Correctness: Composition



$$\frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'}$$

- We assume

$$P(\langle c_0, \sigma \rangle \longrightarrow \sigma'') \triangleq \mathcal{C}[\![c_0]\!]\sigma = \sigma''$$

$$P(\langle c_1, \sigma'' \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_1]\!]\sigma'' = \sigma'$$

- We want to prove $P(\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_0; c_1]\!]\sigma = \sigma'$. We have:

$$\begin{aligned} \mathcal{C}[\![c_0; c_1]\!]\sigma &= \mathcal{C}[\![c_1]\!]^*(\mathcal{C}[\![c_0]\!]\sigma) && [\text{Definition of } \mathcal{C}[\![\cdot]\!]] \\ &= \mathcal{C}[\![c_1]\!]^*\sigma'' && [\text{IH (1)}] \\ &= \mathcal{C}[\![c_1]\!]\sigma'' && [\sigma'' \neq \perp, \text{ by IH (1)}] \\ &= \sigma' && [\text{IH (2)}] \end{aligned}$$

Correctness: If True



$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c_0, \sigma \rangle \longrightarrow \sigma'}{\langle \texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1, \sigma \rangle \longrightarrow \sigma'}$$

- We assume $\langle b, \sigma \rangle \longrightarrow \texttt{tt}$ and therefore $\mathcal{B}[\![b]\!]\sigma = \texttt{tt}$, by the equivalence of the operational and denotational semantics for boolean expressions. Also:

$$P(\langle c_0, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_0]\!]\sigma = \sigma'$$

Correctness: If True



$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c_0, \sigma \rangle \longrightarrow \sigma'}{\langle \texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1, \sigma \rangle \longrightarrow \sigma'}$$

- We assume $\langle b, \sigma \rangle \longrightarrow \texttt{tt}$ and therefore $\mathcal{B}[\![b]\!]\sigma = \texttt{tt}$, by the equivalence of the operational and denotational semantics for boolean expressions. Also:

$$P(\langle c_0, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![c_0]\!]\sigma = \sigma'$$

- To prove: $P(\langle \texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1]\!]\sigma = \sigma'$.
We have:

$$\begin{aligned} \mathcal{C}[\![\texttt{if } b \texttt{ then } c_0 \texttt{ else } c_1]\!]\sigma &= \mathcal{B}[\![b]\!]\sigma \blacktriangleright (\mathcal{C}[\![c_0]\!]\sigma ; \mathcal{C}[\![c_1]\!]\sigma) && \text{[Definition of } \mathcal{C}[\![\cdot]\!]] \\ &= \texttt{tt} \blacktriangleright (\mathcal{C}[\![c_0]\!]\sigma ; \mathcal{C}[\![c_1]\!]\sigma) && \text{[By assumption]} \\ &= \mathcal{C}[\![c_0]\!]\sigma \\ &= \sigma' && \text{[IH]} \end{aligned}$$

Correctness: While False



$$\frac{\langle b, \sigma \rangle \longrightarrow \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma}$$

- ▶ We assume $\langle b, \sigma \rangle \longrightarrow \text{ff}$ and therefore $\mathcal{B}[\![b]\!]\sigma = \text{ff}$
- ▶ Property: $\mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma$ is the fixpoint of $\Gamma_{b,c}$.

Correctness: While False



$$\frac{\langle b, \sigma \rangle \longrightarrow \text{ff}}{\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma}$$

- ▶ We assume $\langle b, \sigma \rangle \longrightarrow \text{ff}$ and therefore $\mathcal{B}[\![b]\!]\sigma = \text{ff}$
- ▶ Property: $\mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma$ is the fixpoint of $\Gamma_{b,c}$.
- ▶ To prove: $P(\langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma) \triangleq \mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma = \sigma$. We have:

$$\begin{aligned}\mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma &= \mathcal{B}[\![b]\!]\sigma \blacktriangleright (\mathcal{C}[\![\text{while } b \text{ do } c]\!]^*(\mathcal{C}[\![c]\!]\sigma); \sigma) && \text{[Definition of } \mathcal{C}[\![\cdot]\!]] \\ &= \text{ff} \blacktriangleright (\mathcal{C}[\![\text{while } b \text{ do } c]\!]^*(\mathcal{C}[\![c]\!]\sigma); \sigma) && \text{[By assumption]} \\ &= \sigma\end{aligned}$$

Correctness: While True



$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \texttt{while } b \texttt{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \texttt{while } b \texttt{ do } c, \sigma \rangle \longrightarrow \sigma'}$$

- We assume $\langle b, \sigma \rangle \longrightarrow \texttt{tt}$ and therefore $\mathcal{B}[\![b]\!]\sigma = \texttt{tt}$. Also:

$$P(\langle c, \sigma \rangle \longrightarrow \sigma'') \triangleq \mathcal{C}[\![c]\!]\sigma = \sigma''$$

$$P(\langle \texttt{while } b \texttt{ do } c, \sigma'' \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]\sigma'' = \sigma'$$

- To prove: $P(\langle \texttt{while } b \texttt{ do } c, \sigma \rangle \longrightarrow \sigma') \triangleq \mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]\sigma = \sigma'$.

Correctness: While True (contd.)



$$\frac{\langle b, \sigma \rangle \longrightarrow \texttt{tt} \quad \langle c, \sigma \rangle \longrightarrow \sigma'' \quad \langle \texttt{while } b \texttt{ do } c, \sigma'' \rangle \longrightarrow \sigma'}{\langle \texttt{while } b \texttt{ do } c, \sigma \rangle \longrightarrow \sigma'}$$

We have:

$$\begin{aligned} \mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]\sigma &= \mathcal{B}[\![b]\!]\sigma \blacktriangleright (\mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]^*(\mathcal{C}[\![c]\!]\sigma); \sigma) && \text{[Definition of } \mathcal{C}[\![\cdot]\!]] \\ &= \texttt{tt} \blacktriangleright (\mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]^*(\sigma''); \sigma) && \text{[By assumption and IH]} \\ &= \mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!]^*(\sigma'') && \text{[Conditional]} \\ &= \mathcal{C}[\![\texttt{while } b \texttt{ do } c]\!](\sigma'') && \text{[By IH } \sigma'' \neq \perp] \\ &= \sigma' && \text{[By IH]} \end{aligned}$$

Completeness: Skip



- To prove:

$$P(\text{skip}) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\text{skip}]\sigma = \sigma' \Rightarrow \langle \text{skip}, \sigma \rangle \longrightarrow \sigma')$$

- By definition we have $\mathcal{C}[\text{skip}]\sigma = \sigma$.

Also, by the operational semantics we have $\langle \text{skip}, \sigma \rangle \longrightarrow \sigma$.

Completeness: Assignment



► To prove:

$$P(x := a) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![x := a]\!]\sigma = \sigma' \Rightarrow \langle x := a, \sigma \rangle \longrightarrow \sigma')$$

Completeness: Assignment



- To prove:

$$P(x := a) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![x := a]\!]\sigma = \sigma' \Rightarrow \langle x := a, \sigma \rangle \longrightarrow \sigma')$$

- By definition of $\mathcal{C}[\![\cdot]\!]$ we have $\sigma' = \sigma[\mathcal{A}[\![a]\!]\sigma/x]$.
- Also, by the equivalence between operational and denotational semantics for expressions we have $\langle a, \sigma \rangle \longrightarrow \mathcal{A}[\![a]\!]\sigma$.
- We can then apply rule

$$\frac{\langle a, \sigma \rangle \longrightarrow n}{\langle x := a, \sigma \rangle \longrightarrow \sigma[n/x]}$$

to conclude $\langle x := a, \sigma \rangle \longrightarrow \sigma[\mathcal{A}[\![a]\!]\sigma/x]$.

Completeness: Composition



► Assume

$$P(c_0) \triangleq \forall \sigma, \sigma'' \in \mathbb{M}. (\mathcal{C}[\![c_0]\!] \sigma = \sigma'' \Rightarrow \langle c_0, \sigma \rangle \longrightarrow \sigma'')$$

$$P(c_1) \triangleq \forall \sigma'', \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_1]\!] \sigma'' = \sigma' \Rightarrow \langle c_1, \sigma'' \rangle \longrightarrow \sigma')$$

► To prove: $P(c_0; c_1) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_0; c_1]\!] \sigma = \sigma' \Rightarrow \langle c_0; c_1, \sigma \rangle \longrightarrow \sigma')$

Completeness: Composition



- Assume

$$P(c_0) \triangleq \forall \sigma, \sigma'' \in \mathbb{M}. (\mathcal{C}[\![c_0]\!]\sigma = \sigma'' \Rightarrow \langle c_0, \sigma \rangle \longrightarrow \sigma'')$$

$$P(c_1) \triangleq \forall \sigma'', \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_1]\!]\sigma'' = \sigma' \Rightarrow \langle c_1, \sigma'' \rangle \longrightarrow \sigma')$$

- To prove: $P(c_0; c_1) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_0; c_1]\!]\sigma = \sigma' \Rightarrow \langle c_0; c_1, \sigma \rangle \longrightarrow \sigma')$

- By assumption we have

$$\begin{aligned} \mathcal{C}[\![c_0; c_1]\!]\sigma &= \mathcal{C}[\![c_1]\!]*(\mathcal{C}[\![c_0]\!]\sigma) \\ &= \mathcal{C}[\![c_1]\!](\mathcal{C}[\![c_0]\!]\sigma) \end{aligned} \quad [\text{Because } \sigma' \neq \perp]$$

- Therefore, $\mathcal{C}[\![c_0]\!]\sigma = \sigma''$ (for some $\sigma'' \neq \perp$) and $\mathcal{C}[\![c_1]\!]\sigma'' = \sigma'$.
- By applying IH twice, we then have $\langle c_0, \sigma \rangle \longrightarrow \sigma''$ and $\langle c_1, \sigma'' \rangle \longrightarrow \sigma'$.
- We use rule
$$\frac{\langle c_0, \sigma \rangle \longrightarrow \sigma'' \quad \langle c_1, \sigma'' \rangle \longrightarrow \sigma'}{\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'}$$
 to conclude $\langle c_0; c_1, \sigma \rangle \longrightarrow \sigma'$.

Completeness: If Then Else



- Assume

$$P(c_0) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_0]\!] \sigma = \sigma' \Rightarrow \langle c_0, \sigma \rangle \longrightarrow \sigma')$$

$$P(c_1) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c_1]\!] \sigma = \sigma' \Rightarrow \langle c_1, \sigma \rangle \longrightarrow \sigma')$$

- We must prove:

$$\begin{aligned} P(\text{if } b \text{ then } c_0 \text{ else } c_1) &\triangleq \\ \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!] \sigma = \sigma' &\Rightarrow \\ \langle \text{if } b \text{ then } c_0 \text{ else } c_1, \sigma \rangle &\longrightarrow \sigma') \end{aligned}$$

- The proof is mostly as before, using the definition of $\mathcal{C}[\![\text{if } b \text{ then } c_0 \text{ else } c_1]\!] \sigma$, with sub-cases $\mathcal{B}[\![b]\!] \sigma = \text{tt}$ and $\mathcal{B}[\![b]\!] \sigma = \text{ff}$.

Completeness: While



- Assume

$$P(c) \triangleq \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![c]\!]\sigma = \sigma' \Rightarrow \langle c, \sigma \rangle \longrightarrow \sigma')$$

- We must prove:

$$\begin{aligned} P(\text{while } b \text{ do } c) &\triangleq \\ \forall \sigma, \sigma' \in \mathbb{M}. (\mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma = \sigma' &\Rightarrow \langle \text{while } b \text{ do } c, \sigma \rangle \longrightarrow \sigma') \end{aligned}$$

- The proof relies on a property of monotone and continuous fixpoints:

$$\mathcal{C}[\![\text{while } b \text{ do } c]\!]\sigma = \text{fix } \Gamma_{b,c} \sigma = \left(\bigsqcup_{n \in \mathbb{N}} \Gamma_{b,c}^n \perp \right) \sigma$$

↑ Kleene's fixpoint theorem

Summary



We have seen IMP and a number of different concepts:

- ▶ Big-step operational semantics
- ▶ Denotational semantics
- ▶ Properties: termination (✗) and determinacy (✓)
- ▶ Well-founded induction: structural induction and rule induction
- ▶ Consistency of operational and denotational semantics

Next lecture: HOFL.



The End